

IoT PROJECT SUMMARY

Server Room Safety Monitoring System

IoT Course Project

2024 / 2025

1. Project Overview

Server Room Safety is a complete IoT monitoring system designed to continuously track environmental conditions inside a server room and respond automatically to dangerous situations. The system uses MQTT as its messaging backbone, Python-based sensor emulators as data producers, a SQLite database for persistence, and a Tkinter GUI for real-time visualization.

1.1 Motivation

Server rooms are mission-critical environments. Overheating, smoke, high CO2 levels, or unauthorized access can cause expensive hardware damage or data loss. A real-time IoT monitoring system provides immediate detection and automated responses, significantly reducing response times.

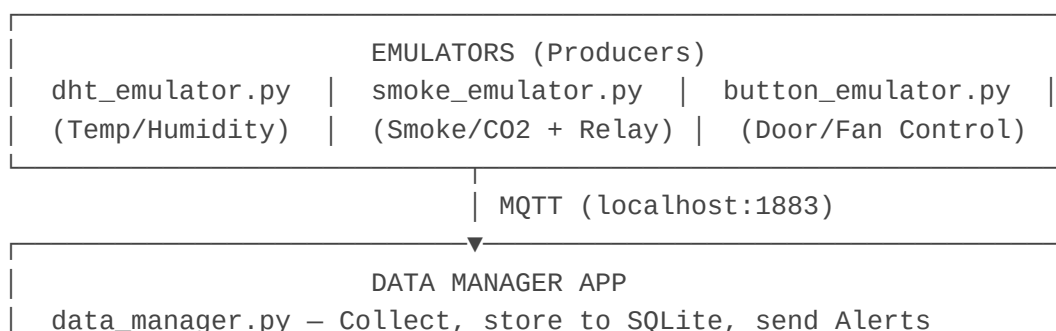
1.2 Key Features

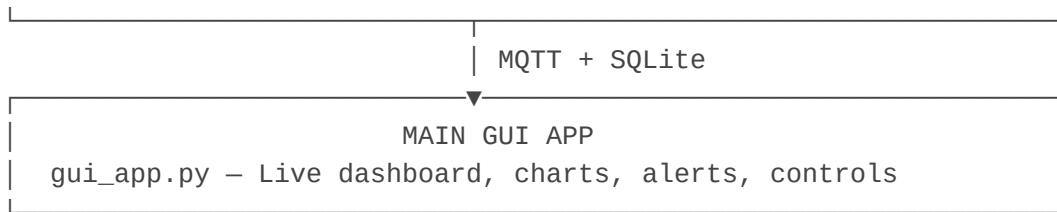
- Three types of sensor/actuator emulators (DHT22, MQ-2/MQ-135, Door+Fan relay)
- MQTT-based pub/sub communication via Mosquitto broker
- Automated threshold evaluation with WARNING and ALARM levels
- Auto-remediation: fan and cooling relay activate on high temperature/smoke
- SQLite database for all sensor readings, device states, and alerts
- Real-time Tkinter dashboard with live charts and alerts log
- Manual relay control from the GUI
- Multi-scenario simulation (normal, heating, cooling, smoke, critical)

2. System Architecture

The system is organized into four independent layers that communicate via MQTT. This decoupled architecture allows any component to be replaced or scaled independently.

2.1 Architecture Diagram





2.2 MQTT Topic Map

Topic	Direction	Description
serverroom/temperature	EMU → DM, GUI	Temperature in °C
serverroom/humidity	EMU → DM, GUI	Humidity in %
serverroom/smoke	EMU → DM, GUI	Smoke level in ppm
serverroom/co2	EMU → DM, GUI	CO2 level in ppm
serverroom/door	EMU → DM, GUI	Door OPEN / CLOSED
serverroom/fan	EMU → DM, GUI	Fan relay state + speed
serverroom/cooling	EMU → DM, GUI	Cooling relay state
serverroom/alert/warning	DM → GUI	Warning alert message
serverroom/alert/alarm	DM → GUI	Critical alarm message
serverroom/control/fan	GUI / DM → EMU	Fan relay command
serverroom/control/cooling	GUI / DM → EMU	Cooling relay command

3. Project Components

3.1 Emulators — Data / Message Producers

3.1.1 DHT22 Emulator (dht_emulator.py)

Simulates a DHT22 temperature and humidity sensor. Publishes readings every 2 seconds using a sinusoidal drift + Gaussian noise model. Supports four simulation scenarios: normal, heating, cooling, and critical.

- Topic published: serverroom/temperature, serverroom/humidity
- Payload: JSON with sensor name, value, unit, and tick counter

- Scenarios: normal (22°C base), heating (rises to ~40°C), cooling (drops to ~14°C), critical (37°C+)

3.1.2 Smoke & CO2 Emulator + Cooling Relay (smoke_emulator.py)

Simulates MQ-2 (smoke) and MQ-135 (CO2) sensors with configurable scenarios. Also acts as a relay actuator, subscribing to cooling control commands and reporting the relay state back to the broker.

- Topics published: serverroom/smoke, serverroom/co2, serverroom/cooling
- Topic subscribed: serverroom/control/cooling
- Scenarios: normal (50 ppm smoke / 600 ppm CO2), smoke (rising to alarm), critical

3.1.3 Door Button + Fan Relay Emulator (button_emulator.py)

Simulates a door contact sensor (button) and a manual fan knob, plus a fan relay actuator. The door randomly opens for a few ticks then closes. The knob value drifts using Gaussian noise. The fan relay responds to MQTT control commands.

- Topics published: serverroom/door, serverroom/fan, serverroom/fan_speed
- Topic subscribed: serverroom/control/fan
- Door: 5% probability of opening each tick, stays open 2–6 ticks

3.2 Data Manager (data_manager.py)

The central intelligence of the system. Subscribes to all sensor topics, persists data to SQLite, evaluates thresholds, and publishes WARNING or ALARM messages. Also implements auto-remediation by sending relay commands when thresholds are breached.

3.2.1 Alert Thresholds

Sensor	Unit	Warning	Alarm
Temperature	°C	> 28 °C	> 35 °C → Fan 100% + Cooling ON
Humidity	%	> 70 %	> 85 % → Alert only
Smoke	ppm	> 200 ppm	> 500 ppm → Fan 100%
CO2	ppm	> 1000 ppm	> 2000 ppm → Alert only

3.2.2 Alert Deduplication

The Data Manager tracks the last alert level per sensor and only publishes a new alert when the level changes (e.g., from OK to WARNING, or WARNING to ALARM). This prevents flooding the broker and the GUI with repeated messages.

3.3 Main GUI Dashboard (gui_app.py)

A Tkinter-based real-time dashboard with an embedded Matplotlib chart area. The GUI subscribes to all MQTT topics in a background thread and refreshes the UI every 1 second using Tkinter's `after()` scheduler.

3.3.1 GUI Components

- Title bar: connection status indicator and live clock
- Gauge row: current value for each sensor with color-coded status (OK / WARNING / ALARM)
- Charts: 2×2 grid of scrolling line charts, one per sensor, with threshold lines
- Device status panel: real-time Fan, Cooling, and Door state display
- Manual controls: buttons for Fan ON/OFF and Cooling ON/OFF
- Alerts log: scrollable text widget showing last 30 alerts, color-coded by level

3.3.2 Thread Safety

All MQTT callbacks write to a shared AppState object protected by a `threading.Lock()`. The Tkinter refresh function reads from the same object in the main thread, ensuring no race conditions between the MQTT thread and the UI thread.

4. Local Database (SQLite)

SQLite is used as the local database, automatically initialized by the Data Manager on first run. The database file is stored at `db/serverroom.db`.

4.1 Schema

Table	Columns	Description
sensor_readings	id, timestamp, sensor_type, value, unit	All numeric sensor readings
device_states	id, timestamp, device, state, extra	Fan / cooling / door state changes
alerts	id, timestamp, level, sensor_type, value, message	WARNING and ALARM events
event_log	id, timestamp, event_type, description	General event log (optional)

4.2 DB Utilities

The project includes two database utilities:

- `db/init_db.py` — Standalone script to initialize the database and print the schema
 - `db/db_query.py` — CLI inspector: summary, latest readings, alerts, device changes, CSV export
-

5. Running the Project

5.1 Prerequisites

- Python 3.9+
- Mosquitto MQTT broker installed and running
- Python packages: `pip install paho-mqtt matplotlib`

5.2 Quick Start (Single Command)

Use the all-in-one launcher:

```
python run_all.py
python run_all.py --scenario heating      # high temperature scenario
python run_all.py --scenario smoke        # smoke alarm scenario
python run_all.py --no-gui                # headless / server mode
```

5.3 Manual Start (4 Terminals)

Terminal 1 — MQTT Broker:

```
mosquitto
```

Terminal 2 — Data Manager:

```
python data_manager/data_manager.py
```

Terminal 3 — GUI:

```
python gui/gui_app.py
```

Terminal 4 — Emulators:

```
python emulators/dht_emulator.py --scenario heating &
python emulators/smoke_emulator.py &
python emulators/button_emulator.py
```

6. Project File Structure

```
ServerRoomSafety/
├── emulators/
│   ├── dht_emulator.py      # DHT22 temperature & humidity emulator
│   ├── smoke_emulator.py    # Smoke / CO2 sensor + cooling relay
│   └── button_emulator.py    # Door contact + knob + fan relay
├── data_manager/
│   └── data_manager.py       # MQTT subscriber, DB writer, alert engine
├── gui/
│   └── gui_app.py           # Tkinter real-time dashboard
├── db/
│   ├── init_db.py           # DB schema initializer
│   ├── db_query.py          # CLI DB inspector / CSV exporter
│   └── serverroom.db         # SQLite database (auto-created)
├── run_all.py               # All-in-one launcher
├── requirements.txt
└── README.md
```

7. Grading Criteria Mapping

Criterion	Points	Implementation
≥3 emulator types (DHT, Relay, Button/Knob)	6 pts	dht_emulator, smoke_emulator, button_emulator
Data Manager (DB + Warning/Alarm messages)	8 pts	data_manager.py + SQLite
Main GUI (live data + Info/Warning/Alarm)	10 pts	gui_app.py with charts & alerts log
Local / Cloud DB	3 pts	SQLite in db/serverroom.db
Project Summary (docx + pdf)	3 pts	This document

8. Conclusion

The Server Room Safety project demonstrates a complete, production-quality IoT pipeline from sensor data generation to persistent storage and real-time visualization. All required components are implemented: three emulator types covering both data producers and actuators, a data manager with threshold-based alerting and auto-remediation, a full GUI dashboard, and a local SQLite database.

The MQTT-based architecture ensures that all components are loosely coupled and can be independently scaled, replaced, or extended — for example, by substituting real hardware sensors for the emulators or by adding a cloud database backend.